



OPTATIVA Machine Learning

Autor/a: Núria Pujol Vilanova | Curs: 2025-2026

Institut TIC Barcelona

Estructures Seqüencials

Les llistes (`list`) i tuples (`tuple`) són estructures de dades seqüencials (seqüències ordenades) fonamentals en Python.

Permeten guardar col·leccions d'elements de qualsevol tipus, mantenint l'ordre d'inserció.

Les principals similituds i diferències entre llistes i tuples:

Característica	Llista	Tupla
Ordenada	Sí	Sí
Indexada	Sí	Sí
Aniuable	Sí	Sí
Mutable	Sí	No
Tipus de dades	Homogènies	Heterogènies

- **Ordenació:** Els elements tenen una posició fixa (índex).
- **Indexació:** Podem accedir a un element concret `seq[índex]`.
- **Mutabilitat:** Les llistes es poden modificar, en canvi les tuples no.

```
noms = ['Maria', 'Marta', 'Albert']  
noms[1] = "Roger" # ['Maria', 'Roger', 'Albert']  
noms.append("Diana") # ['Maria', 'Roger', 'Albert', 'Diana']
```

```
dades = (20, 8, 7.8, 'patata')  
dades[1] = 21 # Obtindrem un error
```

- **Tipus d'elements:** Les llistes solen ser **homogènies** (contenen elements del mateix tipus) mentre que les tuples acostumen a ser **heterogènies** (contenen elements de diferents tipus) però ambdues estructures admeten combinar tipus heterogenis dins la mateixa seqüència.

Constructors

Constructor per crear/definir llistes `[]`:

```
colors = ["vermell", "verd", "blau"]
```

Constructor per crear/definir tuples `()`:

```
persona = ("Maria", 30, "Barcelona")
```

També podem crear llistes o tuples a través de la conversió de tipus amb `list()` o `tuple()`:

```
In [ ]: llista_digits = list(range(5))
        print(llibra_digits)
```

```
In [ ]: llista = [1,2,4]
        tupla_digits2 = tuple(llibra)
        print(tupla_digits2)
```

```
In [ ]: nums = tuple([5, 8])
        print(nums)
```

Inclús podem convertir una cadena de text en una tupla o llista. A continuació es mostra la conversió d'una cadena de text a tupla, permetent accedir als caràcters com a elements individuals però sense poder-los modificar directament.

```
In [ ]: lletres = tuple("Python")
        print(lletres)
```

```
In [ ]: print(lletres[3])
```

```
In [ ]: lletres[3] = 'o'      #Obtindrem un error
```

Podem crear llistes i tuples amb un sol element:

```
In [ ]: llista_1 = ['a']
        print(llibra_1)
```

En el cas de les tuples d'un sol element és important no oblidar-nos de posar la `,`. En cas contrari, Python ho interpretaria com un `int`:

```
In [ ]: tupla_1 = (7, )
        print(tupla_1, type(tupla_1))
```

```
In [ ]: no_tupla_1 = (7)
        print(no_tupla_1, type(no_tupla_1))
```

Estructures aniuades

Podem definir estructures d'estructures, tant llistes com tuples:

```
llibra_anuada = [[1,2], [3,4]]      # Llista de llistes
```

```
info_persona = ("Marc", 32, [95, 88]) # Llista dins de tupla
info_persona = ('Marc', (32,), (95, 88)) #Tuples dins de tupla
```

Operadors sobre seqüències

Les llistes i tuples suporten diversos operadors i funcions per facilitar-ne la manipulació i consulta.

- **Operador de pertinença `in` i `not in`** : comprova si un element es troba dins de la seqüència i retorna un valor booleà: `True` si està present, `False` si no està present.

```
2 in [1, 2, 3]      # True
'a' not in ('x', 'y', 'z') # True
```

- **Concatenació `+`** : uneix dues seqüències del mateix tipus en una nova.

```
[1, 2] + [3, 4]      # [1, 2, 3, 4]
('a',) + ('b',)      # ('a', 'b')
```

- **Repetició `*`** : repeteix la seqüència un nombre de vegades.

```
[0] * 5              # [0, 0, 0, 0, 0]
('x',) * 3            # ('x', 'x', 'x')
```

- **Assignació augmentada `+=`** : combina dues accions en una sola (suma i assignació): enlloc d'escriure `x = x + y`, escrivim `x += y` que fa exactament el mateix però de forma més clara i concisa.

Afegeix a la seqüència original el contingut de l'altra. En tuples, per tal de poder fer l'assignació crea una nova tupla (perquè són immutables).

```
llista = [1, 2]
llista += [3]
# llista = [1, 2, 3]
```

```
tupla = (1, 2)
tupla += (3, 4)
# tupla = (1, 2, 3, 4) # nova tupla amb identitat diferent
```

- **Comparació entre seqüències `==` `!=` `<` `>`** : es compara seqüencialment (elements a element) fins a trobar una diferència que decideixi quin és més gran o si són iguals.

```
[1, 2, 3] == [1, 2, 3]      # True
(1, 5) > (1, 3)              # True
[1, 2] < [1, 3]              # True
```

Quan una seqüència és prefix és més curta però coincideix completament amb els primers elements de l'altra es considera que la seqüència més curta és menor.

```
[1][2] < [1][2][3]      #True
(4, 5) < (4, 5, 6, 7)    #True
```

Funcions comunes sobre seqüències

- `len(seq)`: Retorna la longitud de la seqüència.
- `min(seq)` i `max(seq)`: Retornen el valor mínim i màxim (si es pot comparar).
- `sum(seq)`: Suma dels elements (per números).

```
In [ ]: llista_nums = [5, 3, 8]      #Llista llista_nums
print(len(llista_nums))             # 3
print(min(llista_nums))             # 3
print(max(llista_nums))             # 8
print(sum(llista_nums))             # 16
```

```
In [ ]: tupla_nums = (5, 3, 8)      #Tupla tupla_nums
print(len(tupla_nums))              # 3
print(min(tupla_nums))              # 3
print(max(tupla_nums))              # 8
print(sum(tupla_nums))              # 16
```

Mètodes de seqüències

Degut a la immutabilitat de les tuples els mètodes que podrem aplicar a aquestes seran limitats:

- `count(value)`: retorna el nombre de vegades que podem trobar `value` en la seqüència.
- `index(value)`: retorna la primera posició on apareix `value` en la seqüència.

```
In [ ]: nova_tupla = ("C++", "Python", "Java", "Python", "R", "Python", "Java")
print(nova_tupla.count("Python"))   # 3
print(nova_tupla.index("Java"))     # 2
```

```
In [ ]: nova_llista = [3, 25, 22, 42, 17, 8, 25]
print(nova_llista.count(25))         # 2
print(nova_llista.index(25))         # 1
```

Però si l'element no es troba en la seqüència ens donarà error:

```
In [ ]: print(nova_tupla.index("Perl")) # 2
```

Per tant, és molt típic fer aquest tipus de comprovacions:

```
In [ ]: elements = [10, 20, 30]
element = 20
if element in elements:
    idx = elements.index(element)
else:
    idx = None
print(idx) # 1
```

Les llistes al ser mutables tenen molts més mètodes que ens permeten modificar el seu contingut fàcilment:

- `append(obj)` : afegeix un element al final de la llista.

```
l = [1, 2]
l.append(3)
print(l) # [1, 2, 3]
```

- `extend(seq)` : afegeix nous elements al final d'una seqüència.

```
l = [1, 2]
l.extend([3, 4])
print(l) # [1, 2, 3, 4]
```

- `insert(index,obj)` : insereix un element a una posició indicada, movent la resta d'elements cap a la dreta.

```
l = [1, 3]
l.insert(1, 2)
print(l) # [1, 2, 3]
```

- `remove(obj)` : elimina la primera aparició d'un element.

```
l = [1, 2, 3, 2]
l.remove(2)
print(l) # [1, 3, 2]
```

- `pop(index)` : elimina l'element d'un index específic. Per defecte, si no s'especifica l'index, treu l'últim element de la seqüència.

```
l = [1, 2, 3]
valor = l.pop()
print(valor) # 3
print(l) # [1, 2]
```

```
valor = l.pop(0)
print(valor) # 1
print(l) # [2]
```

- `del seq[index]` : elimina un element de la llista a través d'indicar el seu índex.

```
l = [1, 2]
del l[0]
print(l) # [2]
```

- `clear()` : borra tots els elements.

```
l = [1, 2, 3]
l.clear()
print(l) # []
```

- `sort()` : ordena els elements de la llista in situ, és a dir, modifica la mateixa llista. Té un paràmetre opcional `reverse=True` per ordenar de forma decreixent.

```
l = [2, 1, 3]
l.sort()
print(l) # [1, 2, 3]
```

```
l.sort(reverse=True)
print(l) # [3, 2, 1]
```

- `reverse()` : inverteix l'ordre dels elements dins la mateixa llista, modificant-la.

```
l = [1, 3, 2]
l.reverse()
print(l) # [2, 3, 1]
```

Mètode `copy()` de llistes

Dediquem una secció especial a aquest mètode degut a la importància d'entendre el seu funcionament intern i les conseqüències que implica.

Quan apliquem el mètode `copy()` a una llista, en creem una còpia "superficial".

Aquesta "còpia" consistirà en una nova llista amb els mateixos elements que l'original però associada a una altra identitat de memòria.

Per tant, si modifiquem la "còpia" (afegint o eliminant elements, per exemple), la llista original no es veurà modificada.

```
In [ ]: fruites = ['poma', 'plàtan']
        copia_fruites = fruites.copy()
        copia_fruites.append('kiwi')
        print(fruites)           # ['poma', 'plàtan']
        print(copia_fruites)     # ['poma', 'plàtan', 'kiwi']
```

Si, en lloc de `copy()`, utilitzem l'assignació `nova = original`, totes dues variables apunten a la mateixa llista i els canvis són compartits.

```
In [ ]: nova_fruites = fruites
        nova_fruites.append('kiwi')
        print(fruites)           # ['poma', 'plàtan', 'kiwi']
        print(nova_fruites)     # ['poma', 'plàtan', 'kiwi']
```

Assignació múltiple

L'assignació múltiple a Python permet assignar diversos valors a diverses variables en una sola línia de codi. Per exemple, si tenim una llista o tupla amb diversos elements, podem desempaquetar-la assignant cada element a una variable diferent en ordre.

```
a, b, c = [1, 2, 3]
```

També és possible utilitzar `_` per descartar valors que no volem assignar i també es pot utilitzar l'operador `*` per capturar un nombre variable d'elements restants dins d'una llista.

Fa que el codi sigui més net i flexible perquè estalvia la necessitat d'accedir manualment als índexs o fer assignacions individuals. A més, permet reassignar variables fàcilment, capturar subllistes i ignorar valors innecessaris de manera directa.

```
In [ ]: valors_llista = [10, 20, 30, 40, 50]

# `` serveix per ignorar valors que no t'interessen
a, b, _, _, _ = valors_llista
print(a, b) # 10 20

# ``*` captura múltiples valors restants en una llista
x, *rest = valors_llista
print(x)    # 10
print(rest) # [20, 30, 40, 50]

# També funciona al revés
*rest, y = valors_llista
print(rest) # [10, 20, 30, 40]
print(y)    # 50
```

Quan fas una assignació múltiple amb l'operador `*` a partir d'una tupla, Python desa els valors capturats amb l'asterisc dins d'una llista, encara que l'origen sigui una tupla.

```
In [ ]: valors_tupla = (10, 20, 30, 40, 50)

# `` serveix per ignorar valors que no t'interessen
c, d, _, _, _ = valors_tupla
print(c, d) # 10 20

# ``*` captura múltiples valors restants en una llista
z, *resta = valors_tupla
print(z)    # 10
print(resta) # [20, 30, 40, 50]

# També funciona al revés
*resta, k = valors_tupla
print(resta) # [10, 20, 30, 40]
print(k)     # 50
```